Named Axes as a Systems Discipline: Hybrid Compile-Time and Runtime Coordination for Tensor Programs

[TODO: author block — fill at submission time]
authors@example.com

Abstract

Tensor programs routinely fail on *axis-bug* classes that current type systems cannot catch: silent broadcasts across mis-named dimensions, transposed contractions, and shape-compatible but semantically incorrect reductions. Existing named-axis libraries address this either through runtime string-based notation (`einops`, `einx`) or language-level named tensors with fully runtime checks (`Haliax`, `Penzai`, `xarray`). We argue these approaches under-use the host language's static type system, and present a hybrid design that encodes axis information across the compile-time / runtime boundary. The contributions are: (i) a `_tex` user-defined literal that parses a LaTeX-math subset (e.g. `R"(c_{ij} = a_i b_j)"_tex`) into a named-axis expression AST whose structural indices the type system can reason about, letting the same formula compose with the static-axis machinery at no boundary cost; (ii) a hybrid *NTTP* $\times$ *DynamicShape* axis system that statically rejects axis-name mismatches when shapes are known and falls back to runtime guards otherwise, sharing one kernel implementation across both paths; (iii) a *Hexagonal-lite* kernel-backend port that demonstrates substrate substitutability across a reference implementation, an Eigen-based substrate, and a WebGPU substrate (Dawn) behind a single port boundary. Methodologically, Section 6.1 scores each library on a shared axis-bug benchmark suite (the static catch rate is the distinctive measurement) and Section 6.2 stress-tests substrate substitutability on a bundle-adjustment case study. Numerical results are collected by an open benchmark harness scheduled to land before the camera-ready deadline.

1 Introduction

Consider a researcher writing a multi-head attention block. They compose two linear projections, a softmax, and a matmul. The shapes type-check at every step — every intermediate has the right number of dimensions — yet the network silently produces incorrect outputs at training time, because two axes were transposed five lines up. The type system saw a (B, H, T, D) tensor where the writer meant (B, T, H, D); the names lived only in a comment.

This failure mode is endemic. PyTorch's named-tensor prototype was introduced in 2019 specifically to refute it; six years on it remains labelled "prototype, subject to change" and the community-tracking issue records that development has stalled [16]. JAX's analogous `xmap` was deleted in 2024 [11]. The bug is well-known; what is missing is a design that the host language's type system can refuse, not merely a runtime that throws. Existing libraries respond along two axes:

- **String-based runtime notation** (`einops` [18], `einx` [6]): expressive (Einstein-style formulas are the source of truth) but defers all checking to runtime and forfeits the host language's type system.
- **Language-level named tensors with runtime checks** (`Haliax` [19], `Penzai` [8], `xarray` [10], deprecated `torch.named_tensor` [16]): axis names are first-class values, but verification still happens at runtime.

The thesis. A named-axis library should be able to refute the attention-block bug *at compile time, when the shapes are known*, while still permitting the runtime-shape case that batch processing demands. The test of a design is not whether axis names are present but whether they participate in the type system.

Contributions. This paper presents the `tensor` library, organised around three design moves:

1. **The `_tex` LaTeX bridge** (Section 3): a user-defined literal that parses a LaTeX-math subset (`R"(c_{ij} = a_i b_j)"_tex`) into a named-axis `Expression` AST. The AST's index structure — the names attached to each `IndexedVar` and the bound index on each `Sum` — is what the type system reasons about, so the same formula composes with the static-axis machinery of contribution (ii) at no boundary cost.
2. **Hybrid axis system** (Section 4): non-type template parameters carry compile-time axis sizes; `DynamicShape` carries the runtime-determined cases; the two compose without bifurcating the kernel API.
3. **Hexagonal-lite kernel-backend port** (Section 5): a single port boundary admits three substrates (reference, Eigen, WebGPU-via-Dawn [7]) without leaking substrate concerns into the named-axis layer.

Roadmap. Section 2 surveys the axis-bug taxonomy and the prior-art landscape. Sections 3 to 5 present the three design moves. Section 6 reports the static-catch benchmark and a bundle-adjustment case study across three backends. Section 7 positions against contemporaneous work. Section 8 discusses limitations.

2 Background & Motivation

2.1 The Axis-Bug Taxonomy

We catalogue five recurring axis-bug classes (Table 1). Each class is shape-compatible — the offending program type-checks, runs, and silently produces incorrect output — so a position-typed shape system cannot refute any of them.

2.2 The Landscape

Existing approaches to axis naming differ on two orthogonal axes: *where the axis name lives* and *when it is checked* (Table 2). The space splits into three clusters:

String-based runtime. `einops` [18] and `einx` [6] encode axis intent in formula strings interpreted on each call. The notation is expressive (Einstein-style patterns drive contractions, reshapes, and reductions from one source-of-truth string) but every check fires at runtime and nothing about the formula reaches the host language's type system.

Named-value runtime. `Haliax` [19], `Penzai` [8], `xarray` [10], and the prototype `torch.named_tensor` [16] carry axis names as runtime values attached to each tensor (a `NamedArray` field, an `xarray.DataArray` dimension attribute, the `names=` kwarg). Axis mismatch is caught at the first operation that touches both tensors — the failure mode is no longer "silent wrong output" but "crash at runtime," which is strictly better, yet still misses the opportunity to refuse the program at compile time when the shapes are known. The PyTorch implementation has remained labelled "prototype" since its 2019 introduction and development has stalled [16]; JAX's analogous `xmap` was deleted in 2024 in favour of `shard_map` [11].

Class	One-line reproduction	What is silently wrong
T1. Transposed contraction	`attn = Q @ K.transpose(-2,-1)` where `Q` is laid out `(B,T,H,D)` rather than the expected `(B,H,T,D)`	Contraction sums over the head axis, not over the model dim. Scores have the right shape, wrong meaning.
T2. Silent broadcast	`x + bias.T` where `x` is `(32,128)` and `bias` is `(1,128)`	The transposed bias `(128,1)` stretches against the batch axis instead of the feature axis; every row gets a different bias scalar.
T3. Reduction over wrong axis	`loss = -(targets * x.log()).sum(dim=0)` on logits `x: (B, C)`	Reduces over the batch dimension; the resulting loss vector has length C instead of being a scalar. Loss curve looks "surprisingly small" and training silently diverges.
T4. Layout swap	A module documented as returning `(H, B, D)` is consumed by code expecting `(B, H, D)` and a `.view(B, H, D)` is applied without a `.permute(1,0,2)` first	Adjacent batch entries get mixed across heads; gradients flow incorrectly. The most common production failure mode (PyTorch named tensors [16] were originally introduced to address exactly this).
T5. Stack vs. concat	`torch.cat([a, b], dim=0)` where the intent was to add a new length-2 axis	Produces a tensor of rank r rather than rank $r+1$; downstream `.view` calls then either silently misalign or crash several operators later, far from the original bug site.

Table 1: Axis-bug taxonomy. All five classes are shape-compatible: no positional shape system rejects them. Section 6.1 benchmarks each library's static / runtime / silent rate against an expanded version of this taxonomy.

Type-level (rare). Outside of compiler-IR frameworks (TACO, MLIR `linalg`), only small static-C++ entrants attempt type-level axis names: Einsums encodes index packs as templated literals, Fastor encodes positions as template parameters. Neither admits the runtime-shape case without escape-hatching back to untyped views.

2.3 What remains open

The landscape leaves an explicit gap: *compile-time axis checking that does not abandon the runtime-shape case*. The static approaches in the literature require either fully static shapes (no batch flexibility) or full type erasure at the kernel boundary. Our design target is to recover the latter without giving up the former.

3 Design 1: The `_tex` LaTeX Bridge

3.1 Interface

The user writes the formula the way the corresponding paper would, in a C++ raw-string literal suffixed `_tex`, and feeds it through an Evaluator bound to named tensors:

```
// Bind names -> tensors, then evaluate the formula.
tex::Evaluator<double> ev;
ev.bind("a", DynamicTensor{Shape{Axis{"i", 5}}, /*data*/});
```

Library	Carrier	Check phase
`einops` [18]	string literal	runtime
`einx` [6]	string literal	runtime
`xarray` [10]	runtime value (attribute)	runtime
`Haliax` [19]	runtime value (`NamedArray`)	runtime
`Penzai` [8]	runtime value	runtime
`torch.named_tensor` [16]	runtime value (`names=`)	runtime (stalled)
`tinygrad` [9]	none (positional)	—
`tensor` (this work)	NTTP (`LabelTag<"i">`) *or* runtime `Axis`, per tensor type	**compile time when shapes are static; runtime via the Evaluator otherwise**

Table 2: Where axis names live, and when they are checked. The existing-library cluster is unanimous on runtime checking. The `tensor` library uniquely occupies the compile-time-when-known quadrant without abandoning the runtime case.

```
ev.bind("b", DynamicTensor{Shape{Axis{"j", 5}}, /*data*/});

auto c = ev.evaluate(R"(c_{ij} = a_i b_j)"_tex);
// c has named axes (i, j); c[i,j] == a[i] * b[j].
```

The literal `R"(c_{ij} = a_i b_j)"_tex` parses on construction into an `Expression` AST. The `tex::Evaluator` walks the AST against name-to-tensor bindings, asserting that each `IndexedVar`'s subscript list matches its bound tensor's axis labels in order. For example, `R"(a_{ji})"_tex` bound against a tensor whose axes are declared (i, j) is rejected by the Evaluator's subscript-order check.

3.2 Grammar

The accepted grammar is the LaTeX-math subset useful for naming and combining axis-indexed quantities (informal BNF in priority order):

```
equation    := expression ('=' expression)?
expression  := term (('+' | '-') term)*
term        := factor (('*' | '\cdot' | juxt) factor)*
factor      := ('-' factor) | unary
unary       := atom (('/' atom)*)
atom        := '(' expression ')'
             | '\sum' '_' subscript group
             | name subscript?
subscript   := '_' (letter | '{' letters '}')
group       := '{' expression '}' | atom
```

Whitespace is ignored; juxtaposition is implicit multiplication. The parser is a hand-written recursive descent; the grammar is small enough that the implementation fits in a single header (`include/tensor/tex/parser.hpp`).

The choice of LaTeX over an einops-style mini-language is deliberate. The subset that matters — subscripted variables, Einstein sums (`R"(\sum_i {a_i b_i})"_tex`), and equations that name a result — is the notation tensor-program papers and textbooks already use. The literal is also *readable as LaTeX*, so the same text that drives evaluation can be re-emitted into the program's typeset output: a Jupyter cell prints back the original formula, closing the loop from notation to running code to rendered documentation.

3.3 Composability with the Static-Axis System

The system-level claim of this section is not about *when* the parse fires — it fires at literal construction today, which is runtime — but about *what shape* it produces. The parsed AST's indices are the same structural objects the static-axis machinery of Section 4 reasons about. The Evaluator's binding step is a type-level operation when the bound tensor's axes are NTTP-encoded, and a runtime check when they are carried in a `DynamicShape`. The Evaluator does not branch on which world it is in; one code path services both.

Concretely, the same literal `R"(c_{ij} = a_i b_j)"_tex` drives three different compile-time / runtime mixes without changing form:

```
using namespace tensor::core::literals;       // for "i"_ax, "j"_ax
constexpr auto i = "i"_ax;
constexpr auto j = "j"_ax;

// Scenario (a): both operands NTTP-labelled.
TypedTensor<double, "i"_ax> a_t{{5}, {/*...*/}};
TypedTensor<double, "j"_ax> b_t{{5}, {/*...*/}};
// Bind via to_dynamic(); the label match is a type-level operation
// at template instantiation, sizes are propagated by construction.

// Scenario (b): one NTTP, one DynamicShape (e.g. batch dim from disk).
TypedTensor<double, "i"_ax> a_t2{{5}, {/*...*/}};
DynamicTensor<double> b_d{Shape{Axis{"j", k}}, /*data*/};
// At ev.bind("b", ...): the name 'j' is checked at the call site;
// the runtime extent k is propagated into the Expression's shape.

// Scenario (c): both DynamicShape (e.g. inference on serialised tensors)
    .
DynamicTensor<double> a_d{Shape{Axis{"i", n}}, /*data*/};
DynamicTensor<double> b_d2{Shape{Axis{"j", m}}, /*data*/};
// Both name and size are runtime; both checks fire at ev.bind() and
// at ev.evaluate(...) time. Failure is a ShapeError, not undefined
// behaviour.

tex::Evaluator<double> ev;
ev.bind("a", /* a_t.to_dynamic() or a_d */);
ev.bind("b", /* b_t.to_dynamic() or b_d  */);
auto c = ev.evaluate(R"(c_{ij} = a_i b_j)"_tex);
```

The literal text is identical across (a)–(c). The Evaluator's implementation is identical across (a)–(c). What changes is only *at which moment* each label and each size is verified: at type instantiation, at `bind` time, or at `evaluate` time. The hybrid system pays for the strongest verification available given how much was known at each construction site — and no more.

3.4 Scope and the Scheduled `consteval` Lift

The parser is presently runtime, executed at the literal's construction site. The natural `consteval` formulation is blocked by C++20's prohibition on `std::unique_ptr` escaping a constant-evaluated context (the current AST is a `unique_ptr`-of-variant tree). A scheduled refactor replaces the `unique_ptr` representation with a fixed-size arena-of-variants whose lifetime is the literal's; that representation does escape constant evaluation and lifts the entire parse into the compiler.

Three properties of the current design are unchanged by the lift, and are this section's contribution:

1. the parser's output type (`Expression`) is identical in both worlds;

2. the Evaluator's binding-and-check pass is identical;

3. the parse cost is paid once per literal, not once per call — today at static-initialiser time, after the lift at compile time.

Grammar-wise, the deliberate non-goals are: no ellipsis, no broadcast operators, no rank-polymorphic patterns, no run-time–constructed strings (the literal arrives as `char const*`). Cases the grammar cannot express are carried by the hybrid system of Section 4.

4 Design 2: Hybrid NTTP × DynamicShape

4.1 The two paths

A named tensor carries its axis information in one of two ways:

Compile-time path axis names *and* sizes are encoded as non-type template parameters. A literal expecting axes (i, j) — e.g. `R"(c_{ij} = a_i b_j)"_tex` — bound through the Evaluator against a tensor whose NTTP axes are (i, k) is refuted at template instantiation; the diagnostic names the offending axis.

Runtime path axis names are encoded as template parameters (compile-time) but sizes live in a `DynamicShape` value (runtime). The compiler refutes axis-*name* mismatches; size mismatches surface as a `ShapeError` at the kernel call site.

4.2 Promotion and Demotion

The two paths are explicitly bridged by one demotion entry point (`TypedTensor::to_dynamic()`) and no implicit promotions.

Demotion (CT → RT) is total and free. A `TypedTensor<T, "i"_ax, "j"_ax>` carries its labels as `FixedString` NTTPs and its extents as a static-rank `Shape<2>`. Its `to_dynamic()` method copies the underlying buffer into a `DynamicTensor<T>` whose `DynamicShape` is populated by reading the NTTP labels out at compile time and the extents at runtime — the names *survive the demotion as runtime strings*. The conversion is total: every TypedTensor demotes; nothing is lost except the type-level guarantee.

```
TypedTensor<double, "i"_ax, "j"_ax> t{{2, 3}, /*values*/};
DynamicTensor<double> d = t.to_dynamic();
// d.shape() now has Axis{"i", 2}, Axis{"j", 3}.
// The type information is erased; the *name* information is not.
```

This is what lets a generic Evaluator-driven kernel (Section 3) accept TypedTensor inputs without requiring a templated overload per static-axis pack: the kernel sees one `DynamicTensor` type and the name check uses runtime strings, which the demoted tensor still carries.

Promotion (RT → CT) is opt-in and checked. Going the other direction is not type-safe by construction — the labels read from a `DynamicShape` are runtime data — so the library does not provide an implicit conversion. Instead, the user asserts the expected static labels and the runtime data is verified once:

```
DynamicTensor<double> d = load_from_disk(...);
// d.shape() carries runtime Axis labels and extents.

// Promotion asserts the expected NTTP pack; if the labels do not
// match (or rank differs), construction throws std::invalid_argument
```

```
// at the assertion site rather than silently producing a wrong type.
auto t = TypedTensor<double, "i"_ax, "j"_ax>::from_dynamic(d);
```

This is the only point in the system where the runtime label string is read into a type-level decision. The choice to make promotion explicit means the type system retains a useful invariant: a `TypedTensor<T, L...>` value's labels are always `L...` — always known at compile time, never doubted at runtime.

The two-path rule. A kernel is written exactly once, against the runtime path. Static callers reach it via `to_dynamic()`; dynamic callers reach it directly. The compiler-enforced `static_assert(SameLabels<A,B>)` guard in the `TypedTensor` operators (`operator+`, `operator-`, etc.) refuses static-side mismatches before they reach the demotion call; the Evaluator's name-and-rank checks refuse the same mismatches on the dynamic side. The two checks share the intent ("the labels do not match") and differ only in when they fire.

4.3 Kernel boundary

The contract of a kernel is a function template parameterised by the axis structure (compile-time) accepting `DynamicShape` runtime sizes when needed. *The same kernel* services both paths; the runtime-path overhead is one bounds check per call, not a separate kernel implementation.

5 Design 3: Hexagonal-lite Kernel Backend

The named-axis layer should not know whether contractions are computed by hand-rolled loops, an Eigen template instantiation, or a WGSL kernel dispatched to a GPU. *Hexagonal-lite* is a constrained form of Cockburn's Hexagonal architecture [3]: a single port (`KernelBackend`) with three concrete adapters.

5.1 The Port

The port is a C++20 `concept` declared in `include/tensor/core/concepts.hpp`. A substrate that satisfies `KernelBackend` can be plugged into the Domain without further adaptation work:

```
template <class B>
concept KernelBackend = requires(B b) {
    typename B::backend_tag;
} && requires(B b,
              DynamicTensor<double> const& a,
              DynamicTensor<double> const& a2,
              BroadcastPlan const& bplan,
              ContractPlan const& cplan,
              std::vector<std::size_t> const& source_map,
              DynamicShape const& source_shape) {
    // Element-wise binary, same shape.
    { b.add(a, a2) } -> std::same_as<DynamicTensor<double>>;
    { b.sub(a, a2) } -> std::same_as<DynamicTensor<double>>;
    { b.mul(a, a2) } -> std::same_as<DynamicTensor<double>>;
    { b.div(a, a2) } -> std::same_as<DynamicTensor<double>>;
    // Element-wise unary.
    { b.exp(a)  } -> std::same_as<DynamicTensor<double>>;
    { b.log(a)  } -> std::same_as<DynamicTensor<double>>;
    { b.relu(a) } -> std::same_as<DynamicTensor<double>>;
    { b.neg(a)  } -> std::same_as<DynamicTensor<double>>;
    // Broadcast element-wise.
```

```
    { b.broadcast_add(a, a2, bplan) } -> std::same_as<DynamicTensor<
        double>>;
    { b.broadcast_sub(a, a2, bplan) } -> std::same_as<DynamicTensor<
        double>>;
    { b.broadcast_mul(a, a2, bplan) } -> std::same_as<DynamicTensor<
        double>>;
    // Contraction (Einstein-style).
    { b.contract(a, a2, cplan) } -> std::same_as<DynamicTensor<double>>;
    // Reduction.
    { b.reduce_sum(a) } -> std::same_as<double>;
    // Unbroadcast: used by autograd to reduce dL/dout to an input's
        shape.
    { b.unbroadcast(a, source_map, source_shape) }
        -> std::same_as<DynamicTensor<double>>;
};
```

Three deliberate choices shape this surface:

1. **The Domain pre-computes the plans.** `BroadcastPlan` and `ContractPlan` are computed once in `tensor::core` and handed to the backend, so a substrate never re-implements axis matching or reduction-set inference. Substrates differ on *how* to evaluate a plan, not on *whether* a plan is well-formed.
2. **Unbroadcast is in the port.** The reverse of a broadcasted op — reducing `dL/dout` back to an input's shape — belongs to the substrate because Eigen and Kokkos can implement it as a single fused reduction. Keeping it outside autograd lets the autograd layer compose port methods without per-substrate special cases.
3. **Per-op backward closures are *not* in the port.** They live in `tensor::autograd` and compose port methods. The substrate owns "how to compute," the autograd layer owns "how to differentiate."

Each concrete substrate ends with a `static_assert(KernelBackend<Backend>)`, so any divergence between the port surface and an adapter is a compile error at the point where the adapter is defined, not a runtime surprise.

5.2 Substrates

Reference portable naive nested-loop implementation; defines the semantics that the other substrates must match.

Eigen BLAS-routed contractions for CPU performance baseline; loop fusion via Eigen's expression templates.

WebGPU via Dawn [7] GPU substrate compiled to WGSL; vcpkg-vendored Dawn to keep substrate provisioning reproducible in CI.

The choice of substrate is a CMake variable (`-DTENSOR_KERNEL_BACKEND={reference,eigen,webgpu}`) — the named-axis layer is unchanged across all three.

6 Evaluation

We evaluate two distinct questions: *does the static-axis path actually refuse the axis-bug classes of Section 2.1?* (Section 6.1), and *does the Hexagonal-lite substrate boundary leak performance through its abstraction?*

Library	CT %	RT %	Silent %
`einops` [18]	**[BENCH: ·]**	**[BENCH: ·]**	**[BENCH: ·]**
`einx` [6]	**[BENCH: ·]**	**[BENCH: ·]**	**[BENCH: ·]**
`Haliax` [19]	**[BENCH: ·]**	**[BENCH: ·]**	**[BENCH: ·]**
`Penzai` [8]	**[BENCH: ·]**	**[BENCH: ·]**	**[BENCH: ·]**
`tensor` (ours, static)	**[BENCH: ·]**	**[BENCH: ·]**	**[BENCH: ·]**
`tensor` (ours, dynamic)	**[BENCH: ·]**	**[BENCH: ·]**	**[BENCH: ·]**

Table 3: Static-catch rate per library. Hypothesis: the static `tensor` row's CT column dominates all other rows; the dynamic row matches the named-value-runtime cluster. Numbers pending the harness implementation (see *Status of results* above).

(Section 6.2). The static-catch numbers are the paper's distinctive evaluation; the runtime case study sanity-checks that substrate substitutability does not cost an order of magnitude.

Status of results. Methodology, harness scaffolding, and the comparison protocol are final. Numerical results are collected by a dedicated benchmark harness (`bench/bench_mlsys_paper_case_study.py`), implementation of which is scheduled for September 2026, ahead of the conference deadline; the present submission carries the fully-specified protocols (this section) and an empty result table that the harness output drops into directly. Where numbers below read **[BENCH: ·]**, the design of the measurement is locked but the number itself is pending the harness landing.

6.1 Static-catch Benchmark

Methodology. We construct a benchmark suite of ∼50 small programs, each a self-contained one-screen example exhibiting exactly one bug from Table 1. Each program is replicated across the five libraries that occupy the named-axis cell of Table 2 (`einops`, `einx`, `Haliax`, `Penzai`, and ours) in idiomatic form for that library — e.g. the T1 "transposed contraction" case becomes `einops.rearrange("b h t d -> b t h d", q) @ k.T` for `einops`, an `ein("b h t d -> b t h d", q)` for `einx`, a `NamedArray` contraction in `Haliax`, and the corresponding `_tex` literal applied to TypedTensor inputs in our system.

For each entry, the harness scores the library on three outcomes:

CT (compile time) the program fails to compile (for statically-typed libraries: a `static_assert` or template-instantiation error) or fails at module-load / decorator-application (for Python libraries with at-decoration checks, e.g. `jax.jit`).

RT (runtime, first call) the program raises an exception at the first call to the offending operator (`ShapeError`, `TypeError`, `ValueError`).

Silent the program runs to completion and produces a numerical result that differs from the intended output by more than 10^{-6} in ℓ_∞ norm.

Scoring. The headline figure is the per-library catch rate across the suite:

The `tensor` library appears on *two* rows because its static and dynamic paths offer materially different guarantees; we report both so the cost of opting into the static path is visible.

Backend	t_r (ms)	t_g (ms)	peak (MB)
`tensor` / reference	**[BENCH: ·]**	**[BENCH: ·]**	**[BENCH: ·]**
`tensor` / Eigen	**[BENCH: ·]**	**[BENCH: ·]**	**[BENCH: ·]**
`tensor` / WebGPU (Dawn)	**[BENCH: ·]**	**[BENCH: ·]**	**[BENCH: ·]**
`einops`/`numpy`	**[BENCH: ·]**	**[BENCH: ·]**	**[BENCH: ·]**
`haliax`/JAX	**[BENCH: ·]**	**[BENCH: ·]**	**[BENCH: ·]**

Table 4: Bundle-adjustment case study results across three of our substrates and two cross-library baselines. t_r = time to residual, t_g = time to gradient. Hypothesis: the three `tensor` rows differ by substrate cost only — the named-axis layer adds no measurable overhead. Numbers pending harness landing.

6.2 Bundle-adjustment Case Study

Methodology. We assemble residuals and Jacobians for the standard bundle-adjustment nonlinear least-squares problem on a 50-view $\times$ 200-landmark scene from the BAL dataset [1]. The named-axis structure (`view`, `landmark`, `pose-dof`, `obs`) is invariant across implementations; what varies is the library used to express the Jacobian-assembly contractions and the substrate that evaluates them.

Backends compared. Three substrates of our system, plus two cross-library baselines:

- *`tensor` / reference* — the portable nested-loop reference backend (Section 5.1).
- *`tensor` / Eigen* — the BLAS-routed CPU backend.
- *`tensor` / WebGPU (Dawn)* — the GPU substrate.
- *`einops`* — baseline contracting via NumPy.
- *`Haliax` on JAX* — baseline named-tensor library on the JAX backend.

Metrics. Wall-clock time-to-residual, time-to-gradient, and peak resident-set memory, each averaged over n=10 runs after n=3 warm-up runs. Reproduction substrate: GitHub-hosted runners (Ubuntu 22.04, gcc-11 for the `tensor` substrates; Python 3.11 for the baselines) plus a single consumer GPU (NVIDIA RTX-class) for the WebGPU lane.

A scaling plot of time-to-gradient versus view count ($N_v \in \{10, 25, 50, 100, 200\}$) accompanies the table if page budget allows.

6.3 Threats to Validity

Suite size and curation bias. A $\sim$50-entry static-catch suite is small. We mitigate single-author bias by deriving entries directly from the bug taxonomy (Table 1, each class drawn from documented community reports), not from inspection of any particular library's source. The suite, harness, and per-library idiomatic reproductions ship in the `bench/` directory of the supplementary material so a reviewer can add a class or a library and re-score.

Idiomatic fairness. Each per-library entry is written by someone familiar with the target library's idioms. Mis-classifying a fair use as "Silent" because the writer did not reach for the library's correct guard would inflate our advantage; the harness therefore records, for each entry, a one-line annotation linking to the relevant guidance in the target library's documentation. Where we cannot find such guidance, the entry is annotated "no guard documented" rather than scored.

Hardware variability for the case study. The WebGPU lane runs on a single consumer GPU; absolute numbers are not portable to data-centre hardware. We report the `tensor`-internal cross-substrate ratios as the primary signal and use absolute baseline numbers only to anchor units.

Construct validity of the BA case study. Bundle adjustment exercises high-rank index reasoning (`view` × `landmark`, sparse 2D residual) but under-exercises broadcasting (which is where named-axis libraries diverge most). A second case study sensitive to broadcasting (e.g. multi-head attention forward) would strengthen external validity; we record this as deliberately out of scope for the present 10-page submission and note it in Section 8.

7 Related Work

We position against four clusters: string-encoded notation libraries, runtime-named-value libraries, type-level static-axis libraries, and the underlying notation literature. Table 2 placed each on the (carrier, check-phase) grid; this section adds the qualitative comparison that the grid compresses.

String-encoded notation. `einops` [18] and `einx` [6] pioneered the ergonomic shift from positional to formula-driven tensor manipulation: an `einops.rearrange("b h t d -> b (t h) d", x)` replaces three `.transpose`/`.view` calls with one declarative pattern. `einx` generalises the formula language to a richer set of patterns including stacked broadcasts. Both libraries parse the formula string on every call and reach the host language's type system only as opaque `Tensor`-returning functions; mismatches surface as exceptions, not as compile-time errors. The `tensor` library carries over the design instinct (a literal formula is the right authoring surface) but disagrees on the parse phase: our `_tex` literal exposes its parsed structure to the type system so the static-axis path of Section 4 can act on it. We adopt a LaTeX-math subset rather than einops's bespoke grammar so the same literal can be re-rendered as typeset documentation.

Runtime named values. `Haliax` [19] (Stanford CRFM, JAX-backed), `Penzai` [8] (Google DeepMind, JAX-backed), `xarray` [10] (scientific computing, NumPy/Dask-backed), and the prototype `torch.named_tensor` [16] all promote axis labels from comments to values: the `NamedArray` carries its dimension names as a runtime field, and operators dispatch on label match. This converts T1–T5 of Table 1 from "silent wrong output" to "runtime exception" — a strict improvement. Two structural limits remain: (i) when the tensor's shape *is* statically known, the runtime check is doing avoidable work and emitting an avoidable diagnostic; (ii) the named-tensor lineage has had difficulty stabilising ergonomically (PyTorch's stalled prototype; JAX's deletion of `xmap` [11]), suggesting that runtime-only is not the final design point. Our hybrid system uses the runtime-named approach where shapes genuinely are dynamic and reaches for static-axis types when they are not.

Type-level static axes. Outside compiler frameworks, a small C++ literature attempts type-level axis tracking: `Einsums` [5] encodes Einstein-pattern packs as template arguments; `Fastor` [15] encodes positional axis extents as template parameters and the `einsum` as a formula at the type level; `Tenseur` [13] explores lazy expression templates over BLAS. These libraries reject mismatches at compile time when the shape is fully static, but offer no escape hatch to a runtime-shape regime — a tensor whose rank or extent is learned from disk cannot be expressed without dropping out of the typed API entirely. Our hybrid system shares the static rigour of this cluster on the static path and adds the missing runtime path via `DynamicTensor` + `TypedTensor::from_dynamic` (Section 4.2).

Compiler-IR and DSLs. TACO [12], COMET [4], and MLIR's linalg/tensor dialects [14] express index-name semantics formally, but at the compiler-IR level: the user writes either a target-DSL string or invokes the compiler toolchain rather than calling library-level operators. Halide [17] similarly separates algorithm from schedule. tinygrad [9] keeps the API at twelve primitive ops and infers shapes positionally. These are useful reference points for what "maximal axis discipline" looks like at the codegen level; our work targets the library-API level with no compiler stage.

Notation and formalism. Chiang and Rush [2] formalise named-tensor notation as a typed calculus and present a paper notation usable across linear algebra textbooks and ML papers. We adopt their notation as the authoring surface (Section 3.2 accepts a restricted LaTeX subset compatible with theirs) and supply the type-system implementation that the calculus invites but which their paper does not provide: an in-language host (C++20) where the named-axis structure is what the type-checker reasons about.

8 Discussion and Limitations

When to reach for which path. The hybrid system offers three settings and an upgrade door; the rule of thumb is to take the strongest guarantee that the calling context supports:

- *All shapes and labels known at the call site* (e.g. a fixed model architecture, a layer's weight tensor at module construction): use `TypedTensor<T, L...>`. Label mismatches are refuted at template instantiation; the compiler diagnostic names the offending axis.
- *Labels known statically, sizes vary at runtime* (e.g. the batch dimension): use `TypedTensor` for the labels and let extents be runtime — the static check refuses wrong-name compositions; the runtime check fires on wrong-size compositions. This is the most common case in ML kernels.
- *Labels and sizes both runtime* (e.g. tensors deserialised from disk, user-supplied inputs at inference time): use `DynamicTensor` directly. Both checks fire at the first operator call; the failure mode is a `ShapeError`, not undefined behaviour.
- *Bootstrap into the static path*: when a tensor enters the system as `DynamicTensor` but downstream code benefits from static guarantees, an explicit `TypedTensor::from_dynamic` call verifies the runtime labels against the expected NTTP pack once and crosses the boundary (Section 4.2). After that point all downstream operations use the static path.

Limitations. (L1) UDL grammar is deliberately small. The `_tex` grammar (Section 3.2) accepts subscripted variables, sums, equations, and the usual arithmetic operators. It does not accept rank-polymorphic patterns (einops's `...` ellipsis), no broadcast-aware patterns (einx-style `(t h)`-grouping), and no runtime-constructed formula strings. The hybrid system absorbs the runtime-shape case (Section 4), but the design does not currently express rank-polymorphism as a first-class feature.

(L2) Diagnostic quality on the static path is template-deep. A label mismatch in `TypedTensor<T, "i"_ax> + TypedTensor<T, "j"_ax>` produces a `static_assert` failure with the intended message ("axis labels must match"), but it is wrapped in the operator's template instantiation backtrace. We provide a `assert_same_label` helper that surfaces the relevant labels directly, and the backtrace begins at the operator site rather than deep in the compiler — yet C++'s default error rendering is far from the ergonomic that languages like Rust or Swift have established as the bar. Diagnostic surface is genuine future work.

(L3) WebGPU substrate portability is constrained. The Hexagonal-lite `webgpu` adapter is built atop Dawn [7] and inherits Dawn's platform matrix (Windows, macOS, Linux, ChromiumOS; Android is preview). On Linux, software emulation paths exist but performance numbers should not be extrapolated across them. Substrates beyond reference / Eigen / WebGPU — Kokkos, SYCL, or direct CUDA — are not yet implemented; the port surface (Section 5.1) is designed to receive them without changes.

(L4) Single-author empirical coverage. The static-catch suite and the BA case study were curated by the authors and may reflect blind spots that an external reviewer would catch. The harness in `bench/` is structured to accept contributed entries; the per-entry annotation linking each "Silent" classification to the target library's documentation is intended as a falsification handle for readers.

Future work. The primary scheduled lift is moving the `_tex` parser from runtime (literal-construction time) to `consteval` (compile time) by replacing the `unique_ptr`-of-variant AST with a fixed-size arena representation that escapes constant evaluation (Section 3.4); the output type and Evaluator pass are unchanged, so this is a mechanical refactor whose payoff is moving parse cost from program startup to compile time. Beyond that: extending the grammar towards rank-polymorphic patterns without losing the structural-index property; auto-tuned substrate selection based on shape and target; integration with Triton-class kernel languages at the Hexagonal-lite adapter layer.

9 Conclusion

Named-axis libraries should treat the host language's type system as load-bearing infrastructure, not as a place to attach metadata. The `tensor` library demonstrates that the static and dynamic worlds compose: a LaTeX-bridging `_tex` UDL whose AST shape is the same object the type system reasons about, a hybrid NTTP $\times$ DynamicShape axis system that shares one kernel across both paths, and a Hexagonal-lite kernel backend that demonstrates substrate substitutability across three concrete adapters. The static-axis path is what lets the type system refuse the axis-bug taxonomy of Section 2.1; the runtime path is what keeps the design livable for the cases statically-known shapes do not cover.

References

[1] Sameer Agarwal, Noah Snavely, Steven M. Seitz, and Richard Szeliski. Bundle adjustment in the large. In *European Conference on Computer Vision (ECCV)*, 2010. The BAL dataset `https://grail.cs.washington.edu/projects/bal/`.

[2] David Chiang and Alexander M. Rush. Named tensor notation. In *Transactions on Machine Learning Research*, 2024. Formalises named-axis notation as a typed calculus.

[3] Alistair Cockburn. Hexagonal architecture (Ports and Adapters). `https://alistair.cockburn.us/hexagonal-architecture/`, 2005. Originally published 2005; revised through 2024.

[4] COMET contributors (PNNL). COMET: Domain-specific COMpiler for Extreme-scale Tensor algebra. `https://github.com/pnnl/COMET`, 2021–2026.

[5] Einsums contributors. Einsums: A C++ tensor library with compile-time einstein-notation patterns. `https://einsums.github.io/Einsums/`, 2023–2026.

[6] einx contributors. einx: Universal tensor operations using einstein notation. `https://github.com/fferflo/einx`, 2023–2026.

[7] Google Chrome. Dawn: A WebGPU implementation. https://dawn.googlesource.com/dawn, 2020–2026.

[8] Google DeepMind and contributors. Penzai: A jax research toolkit with named tensors. https://github.com/google-deepmind/penzai, 2024–2026.

[9] George Hotz and tinygrad contributors. tinygrad: A deep-learning framework for the small. https://github.com/tinygrad/tinygrad, 2020–2026.

[10] Stephan Hoyer and Joseph Hamman. xarray: N-D labeled arrays and datasets in Python. *Journal of Open Research Software*, 2017.

[11] JAX maintainers. Deprecation and removal of xmap from JAX. https://github.com/jax-ml/jax/discussions/20312, 2024.

[12] Fredrik Kjolstad, Shoaib Kamil, Stephen Chou, David Lugato, and Saman Amarasinghe. The tensor algebra compiler. In *Proc. ACM Program. Lang. (OOPSLA)*, 2017.

[13] Istvan Marc. On designing Tenseur: a C++ tensor library with lazy evaluation. https://istmarc.github.io/post/2024/10/27/on-designing-tenseur-a-c-tensor-library-with-lazy-evaluation/, 2024.

[14] MLIR project. MLIR linalg Dialect. https://mlir.llvm.org/docs/Dialects/Linalg/, 2020–2026.

[15] Roman Poya and Fastor contributors. Fastor: A lightweight high-performance tensor algebra library for modern C++. https://github.com/romeric/Fastor, 2017–2026.

[16] PyTorch. Named tensors (experimental). https://pytorch.org/docs/stable/named_tensor.html, 2019–2026. API marked experimental since 2019; community-tracking status at https://github.com/pytorch/pytorch/issues/60832.

[17] Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman Amarasinghe. Halide: A language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. In *PLDI*, 2013.

[18] Alex Rogozhnikov. einops: Clear and reliable tensor manipulations with einstein-like notation. https://github.com/arogozhnikov/einops, 2018–2026.

[19] Stanford CRFM and contributors. Haliax: Named tensors for jax. https://github.com/stanford-crfm/haliax, 2023–2026.